

THE TIMES
THE SUNDAY TIMES
GOOD
UNIVERSITY
GUIDE
2022
UNIVERSITY
OF THE YEAR
SHORTLISTED

#WeAreUU

Ulster
University

COM668 Project

“Deep Dive on Requirements Engineering”

Some Useful Requirements Development & Management Techniques

Roy Sterritt

Ulster University

r.sterritt@ulster.ac.uk

[@RoysterUlster](https://twitter.com/RoysterUlster)



Google Scholar



COM668 Project

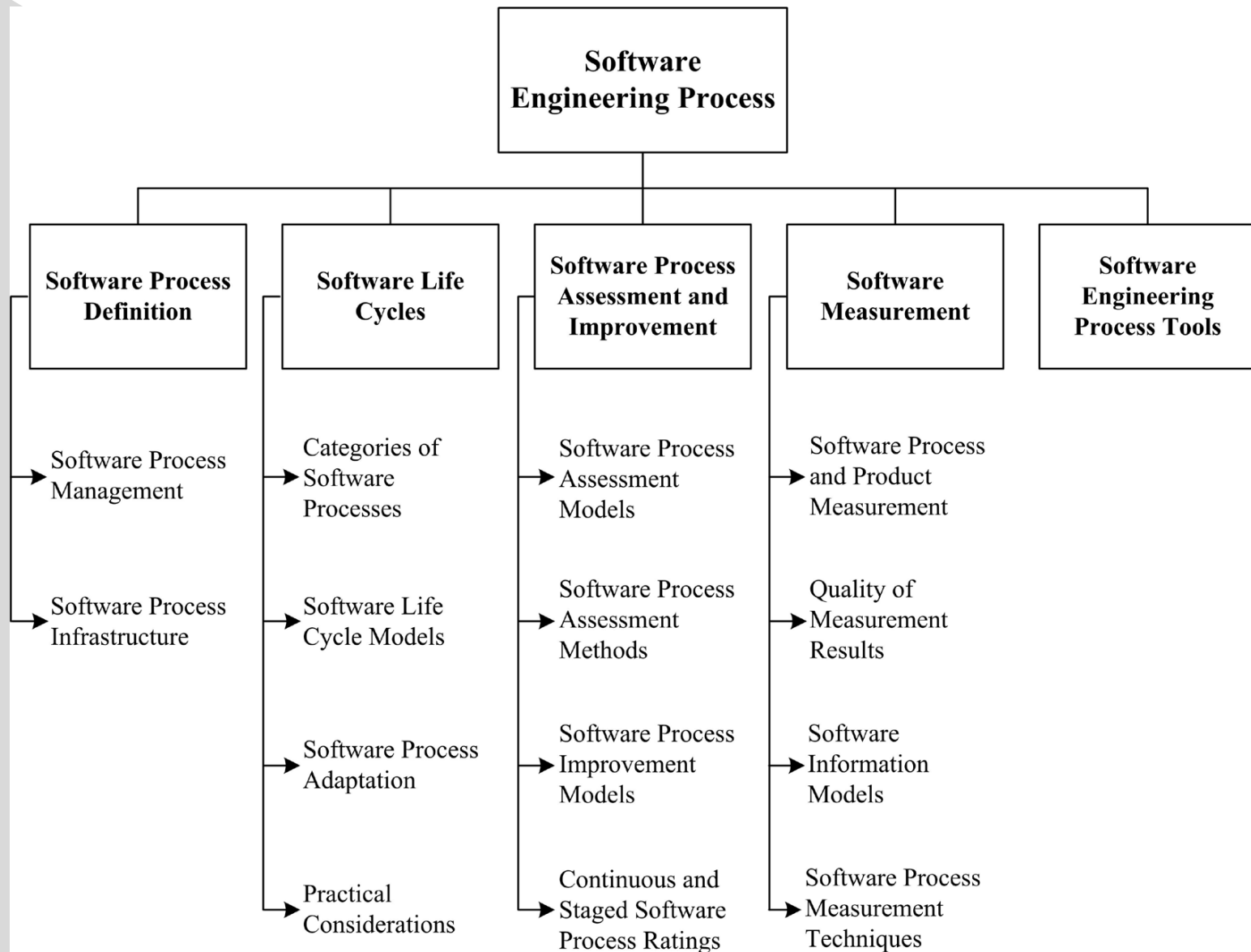
Some Useful Requirements Development & Management Techniques

Requirements Development (RD)

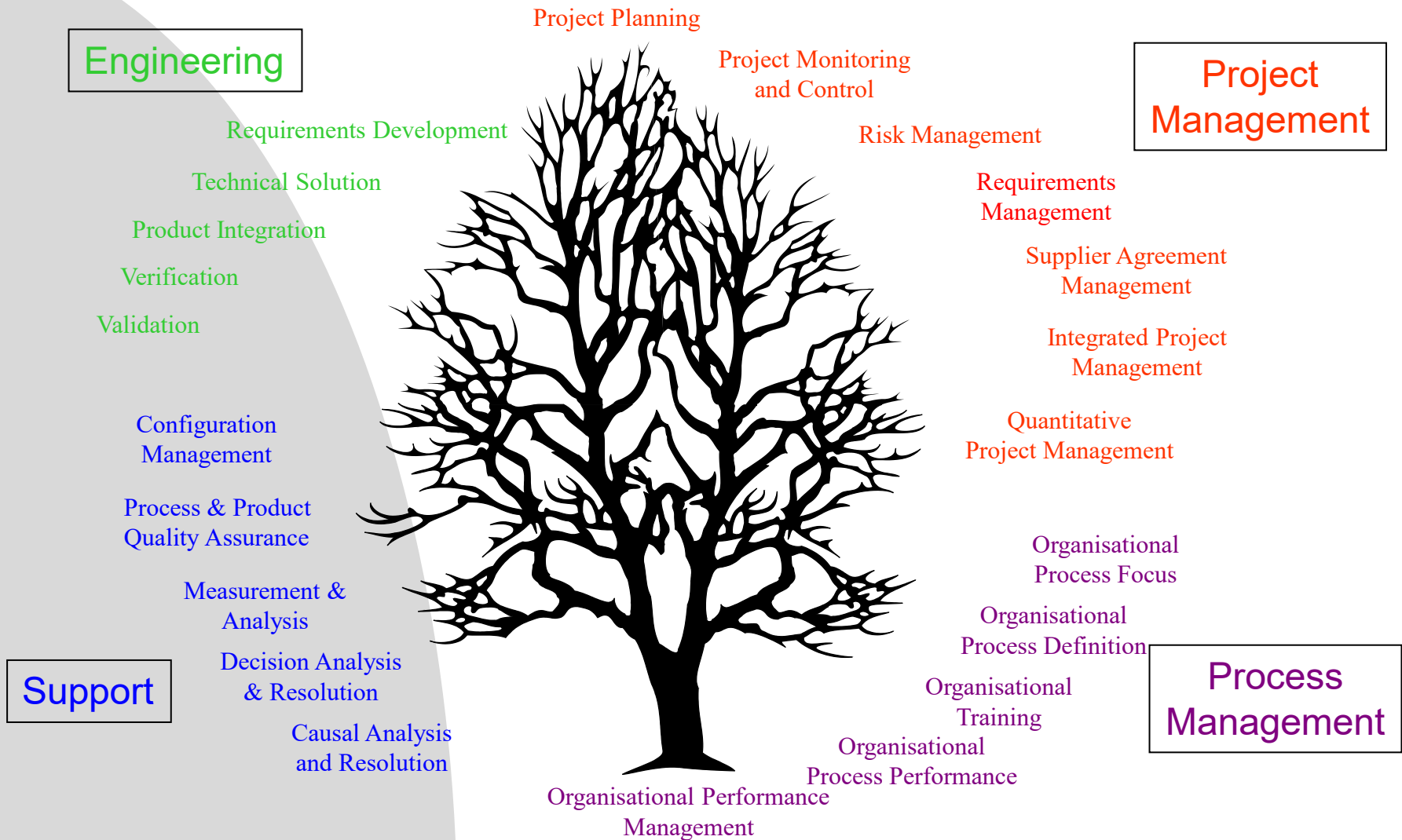
Requirement Management (REQM)

Configuration Management (CM)

SWEBOK® Guide V3.0 2014



CMMI® Process Areas



CMMI Requirements Development (RD)

The purpose of Requirements Development (RD) is to elicit, analyze, and establish customer, product, and product component requirements.

SG 1 Stakeholder needs, expectations, constraints, and interfaces are collected and translated into customer requirements.

- SP 1.1 **Elicit stakeholder needs**, expectations, constraints and interfaces for all phases of the product lifecycle.
- SP 1.2 Transform stakeholder needs, expectations, constraints and interfaces into **prioritized customer requirements**.

SG 2 Customer requirements are refined and elaborated to develop product and product component requirements.

- SP 2.1 Establish and maintain product and product component requirements, which are based on the customer requirements.
- SP 2.2 Allocate the requirements for each product component.
- SP 2.3 Identify interface requirements.

SG 3 The requirements are analyzed and validated.

- SP 3.1 Establish and maintain operational concepts and associated scenarios.
- SP 3.2 Establish and maintain a definition of required functionality and quality attributes.
- SP 3.3 Analyze requirements to ensure that they are **necessary and sufficient**.
- SP 3.4 Analyze requirements to **balance** stakeholder needs and constraints.
- SP 3.5 **Validate requirements** to ensure the resulting product will perform as intended in the end user's environment.

Specifying Requirements

- Requirements are usually written as a means for communication between the different stakeholders. This means that the requirements should be easy to understand both for normal users and for developers. One common way to document a requirement is stating what the system must do. Example: 'The contractor must deliver the product no later than xyz date.' Other methods include use cases and user stories.

Customer Vs Product Requirements

- Product requirements are established in the context of a solution.
- During development there is a movement away from customer requirements to the establishment of product requirements
- Let us consider some examples.
 - User requirement - We want the room to be heated
 - Product Requirement - The room shall have a conventional, thermostatically controlled radiator

Levels of Requirements

- Customer Requirement
 - High level usually non-technical terms used
 - May be functional or non-functional
 - eg. “All information must be managed in adherence with the General Data Protection Regulation (GDPR)”
- Product Requirement
 - “The System must provide secure access for users”
- Derived Requirements
 - These result from analysis of product requirements and allocation to a logical architecture
 - eg. “Voice recognition will be used to verify users”
 - “The user will be required to speak their name only”
 - “As a backup/alternative, the user will be able to enter a password”
 - “Passwords must be not less than 8 characters or more than 32, with at least one capital letter and one numeral”.

Requirements Reviews – Quality Assurance (part of PPQA Process Area)

- Check that the specified requirements conform to the quality criteria
 - Atomic
 - don't combine functions in the one requirement
 - eg. "Login can be facilitated via voice recognition or password" – split into two requirements.
 - Understandable
 - to the customer and developers alike
 - Unambiguous
 - split into several derived requirements if necessary to make clear
 - Testable
 - composing a test at the time of requirement writing is a really good idea
 - Realisable
 - can be implemented
 - Non-redundant
 - Consistent
 - does not contradict any other requirement
 - Traceable
 - back to a business/customer requirement
 - Complete
 - fully stated in one place
- Doing it
- Capable
- Expert

Prioritisation

- The customer's opinion is important here
- But its not the only consideration
 - Functional priority must also be considered
 - eg. You cannot EDIT a customer record *before* you have provided a facility to CREATE such a record in the first place
 - In terms of sprint planning, the backlog items scheduled for a given sprint will usually be logically related, regardless of their individual priorities.
 - Sometimes, the time/effort to implement a feature might also be considered when planning its deployment.
 - Seek customer approval when you alter priorities from their opinion.
 - Remember, there will normally be further product releases.

MoSCoW Method of Prioritisation

- **MUST HAVE**
 - Critical to current delivery
- **SHOULD HAVE**
 - Often not as time-critical
- **COULD HAVE**
 - May improve user experience but not necessary
- **WON'T HAVE (now)**
 - Useful to note good ideas possibly for the future

Prioritisation

- An excellent method for prioritization is ***Relative Weighting***, which Karl Weigers introduced in 1999. This method not only provides a mechanism for prioritizing requirements based on user input and feedback but also includes the expert judgment of the team. Relative Weighting allows the product owner to better gauge which features to implement and in what priority order.

Consider Benefits and Penalties

- The first step is to assign a weight to the relative benefit of a feature. A benefit is the advantage for users of having the feature in the final product.
- Next is to assign the relative penalty. The penalty is the consequence for users of not having the feature in the final product.
- The weights are arbitrary numbers that represent **how your company values benefits and risks of features**. It is very similar to how a teacher determines what weight to give homework, attendance, quizzes, and tests in determining the overall grade; it will vary from teacher to teacher.
- If you decide that the benefits outweigh the penalties, make the weight higher than the penalty by whatever ratio you see fit. If you decide that penalties outweigh benefits, adjust the weighting accordingly.
- In the example in table 2, we gave the benefit a relative weight of 2 and the penalty a relative weight of 1, meaning that the benefit will be a larger factor in determining the final priority.

Consider Cost and Risk

- In the same way, we determine the weight of cost percentage and risk percentage. In the following example, risk was not as much of a concern to the project, so cost percentage was given weight of 1 and risk percentage a weight of 0.5. (Note that, although benefit and penalty are weighted before the value percentage is calculated, cost and risk are weighted as percentages.) If you have a high-risk project, you might weight risk higher than cost.

Requirements Priority Table

[illegible]

Rate the Relative benefits and penalties

- Now that the weights are set, we ask the users to rate each feature's relative benefit and relative penalty.
- Each benefit and penalty is rated on a set scale—Weigers recommends 1-9, but you could choose a different scale as long as you're consistent. **The relative benefit is the value that the feature will deliver; relative penalty is the potential negative impact of not doing the feature.**
- For example, let's say one possible feature is "Make the widget comply with GDPR." There is virtually no relative benefit to users, but the relative penalty to the business is great - *not* doing it could be illegal and put the company out of business. As such, we might see a score of 1 or 2 for the relative benefit and a score of 8 or 9 for the relative penalty.
- In the following example, users rated the relative benefit of the feature "Query status of a vendor order" as a 5. They rated its relative penalty as a 3.
- To derive the total value of that feature, we multiply the two numbers by their relative weights and add them together:
- $(\text{Benefit} * \text{Weight}) + (\text{Penalty} * \text{Weight}) = \text{Total Value}$
 - $(5 * 2) + (3 * 1) = 13$
 - In this case, the total value for the feature is 13 points.

Requirement Priority Table

<i>Relative Weight</i>	2	1			1		.5		
<i>Feature</i>	<i>Relative Benefit</i>	<i>Relative Penalty</i>	<i>Total Value</i>	<i>Value %</i>	<i>Relative Cost</i>	<i>Cost %</i>	<i>Relative Risk</i>	<i>Risk %</i>	<i>Priority</i>
1. Query status of a vendor order	5	3	13						
2. Generate a Chemical Stockroom inventory report	9	7	25						
3. See history of a specific chemical container	5	5	15						
4. Print a chemical safety datasheet	2	1	5						
5. Maintain a list of hazardous chemicals	4	9	17						
6. Modify a pending chemical request	4	3	11						
7. Generate an individual laboratory inventory report	6	2	14						
8. Search vendor catalogs for a specific chemical	9	8	26						
9. Check training database for hazardous chemical training record	3	4	10						
10. Import chemical structures from structure drawing tools	7	4	18						
Totals	54	46	154						

Ask Team to Estimate Cost and Risk

- When we get the total relative value and value percentage for the features, we move away from the users to get insight from the *team*. We ask the **team** to estimate the relative cost to implement each feature using the same scale.
- Developers estimate the cost ratings based on factors such as the requirement complexity, the extent of user interface work required, the potential ability to reuse existing designs or code, and the levels of testing and documentation needed.
- After we estimate relative cost, we estimate relative risk.
- Developers estimate the relative degree of technical or other risk associated with each feature on a scale from 1 to 9. An estimate of 1 means you can program it in your sleep, while 9 indicates serious concerns about feasibility, the availability of staff with the needed expertise, or the use of unproven or unfamiliar tools and technologies.
- The spreadsheet will calculate the percentage of the total risk that comes from each feature.
- After we note the values for relative benefit, penalty, cost, and risk, we sum up each column. These totals will be used to calculate the percentages.

Spreadsheet then calculates...

- **Total value percentage:** Divide the sum value of the relative benefit and penalty by the total sum at the bottom. In the following example, this is $13 / 154 = 8\%$.
- **Relative cost percentage:** Divide the relative cost value by the total relative cost sum at the bottom. In the following example, this is $2 / 42 = 4.8\%$.
- **Relative risk percentage:** Divide the relative risk value by the total relative risk sum at the bottom. In the following example, this is $1 / 33 = 3\%$.
- **Priority:** Divide the value percentage by (cost percentage * cost weight) + (risk percentage * risk weight). In the following example, this would be $8.4\% / ((4.8\% * 1) + (3\% * 0.5))$. This gives the priority value (1.345).

After we obtain the priority values, we sort the priority column in descending order so that the highest priority items are at the top. As items are added to the product backlog or more information about a story emerges, we'll need to reassess priority.

<i>Relative Weight</i>	<i>2</i>	<i>1</i>			<i>1</i>		<i>.5</i>		
<i>Feature</i>	<i>Relative Benefit</i>	<i>Relative Penalty</i>	<i>Total Value</i>	<i>Value %</i>	<i>Relative Cost</i>	<i>Cost %</i>	<i>Relative Risk</i>	<i>Risk %</i>	<i>Priority</i>
1. Query status of a vendor order	5	3	13	8.4	2	4.8	1	3	1.345
2. Generate a Chemical Stockroom inventory report	9	7	25	16.2	5	11.9	3	9.1	0.987
3. See history of a specific chemical container	5	5	15	9.7	3	7.1	2	6.1	0.957
4. Print a chemical safety datasheet	2	1	5	3.2	1	2.4	1	3	0.833
5. Maintain a list of hazardous chemicals	4	9	17	11	4	9.5	4	12.1	0.708
6. Modify a pending chemical request	4	3	11	7.1	3	7.1	2	6.1	0.702
7. Generate an individual laboratory inventory report	6	2	14	9.1	4	9.5	3	9.1	0.646
8. Search vendor catalogs for a specific chemical	9	8	26	16.9	7	16.7	8	24.2	0.586
9. Check training database for hazardous chemical training record	3	4	10	6.5	4	9.5	2	6.1	0.517
10. Import chemical structures from structure drawing tools	7	4	18	11.7	9	21.4	7	21.2	0.365
Totals	54	46	154	100	42	100	33	100	--

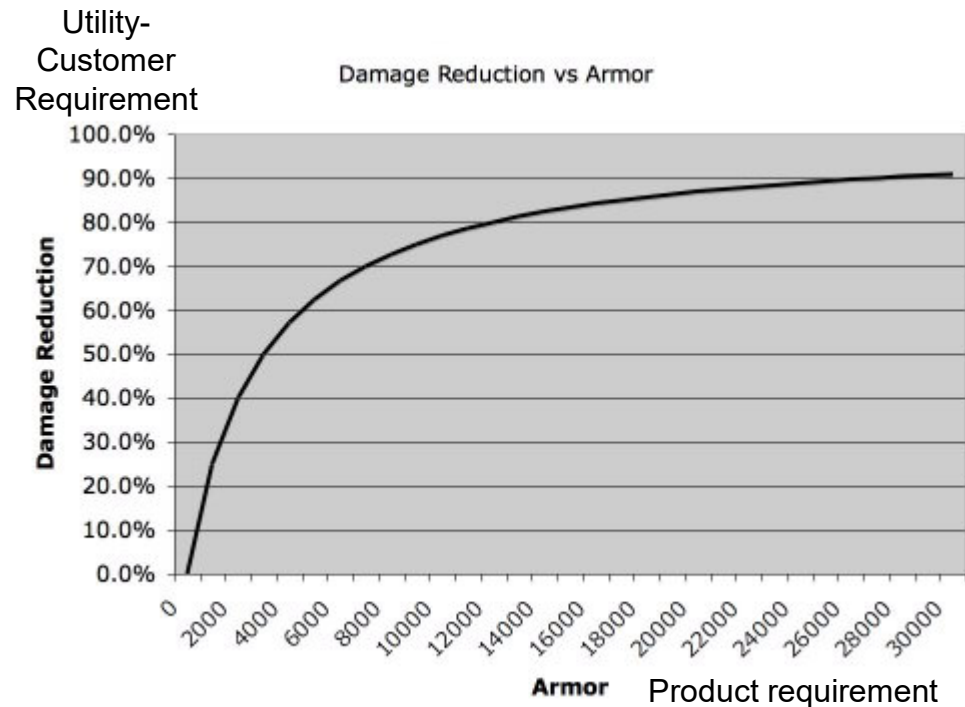
By taking this approach, you can better understand the ranges that work for the team and for the stakeholders. It also helps to better ground discussions because it can be difficult to objectively factor in elements such as benefit, penalty, cost, and risk for each feature.

Kano Analysis

- Kano analysis allows us to prioritize requirements as a function of customer satisfaction
- Kano defined four categories into which each feature or requirement can be classified
 - ***Surprise and delight.*** Capabilities that differentiate a product from its competition (e.g. Samsung curved screen).
 - ***More is better.*** Dimensions along a continuum with a clear direction of increasing utility (e.g. battery life or song capacity).
 - ***Must be.*** Functional barriers to entry—without these capabilities, customers will not use the product (e.g. Regulatory approval).
 - ***Better not be.*** Represents things that dissatisfy customers (e.g. inability to increase song capacity via upgrades).

More is better requirements- Benefit

- Specifying precise objectives can be very difficult as well. The law of diminishing returns comes into play.
- There is a concept in economics called utility which represents the tangible and intangible benefits of something. We can consider the utility of a feature with respect to the target users.
- When writing a requirement, how do we determine the armour that is truly required? It would be ambiguous to say “as protective as possible”



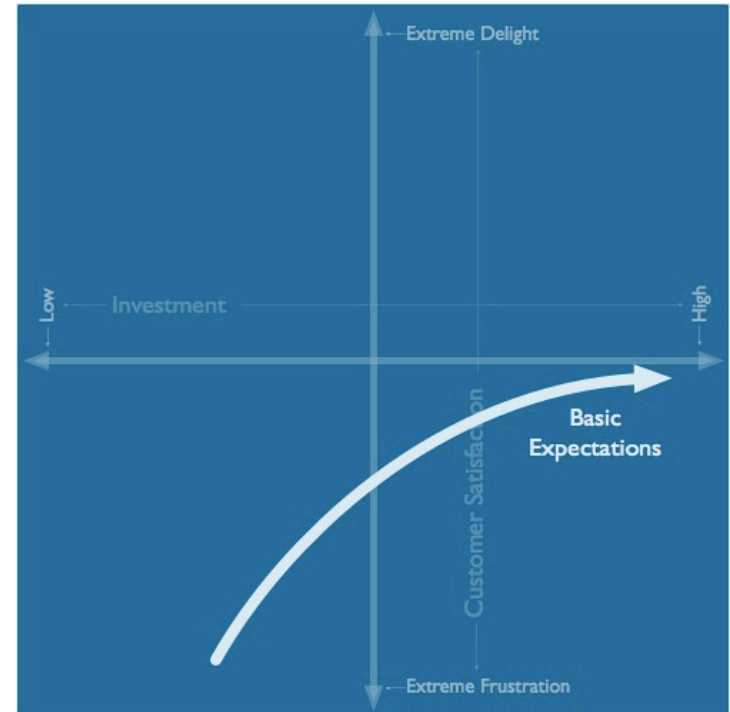
Must Be - Basic Expectations

- Kano challenges the conventional belief that improving every aspect of your product or service leads to increased customer satisfaction, asserting that **improving certain aspects only serves to maintain basic expectations**, whereas improving other aspects can delight customers with less effort.
- Basic features that a user expects to be there and work will **never score highly** on satisfaction, but can take inordinate amounts of effort to build and maintain.
- No matter how much effort you put into this feature you will only ever be able to meet basic expectations. Investment in this area is all about downside protection. This doesn't make them any less important, but it becomes **crucial to understand at what point you've met the users' expectations and to stop there**. Any further development is a wasted effort.
- Modern hotels spend a fortune ensuring that every room has hot water instantly available, night and day. Now, no customer is going to rave on Twitter or TripAdvisor about how good the hot water in their hotel was, but if it wasn't there, or it wasn't hot enough, or took too long to get hot, you bet there would be a complaint, and a loud one.
- One challenge with basic expectations is that **a user will never tell you about them**. If you sat someone down and asked them to design their perfect hotel they'd probably talk about comfortable beds, a nice TV, maybe the view, etc. They would never think to mention that they'd expect to be able to have a hot shower at 3am. You have to observe and research to find these basic expectations.

Kano

Capabilities that users expect will frustrate those users when they don't work. However, when they work well, they don't delight those users. A **basic expectation, at best, can reach a neutral satisfaction** a point where it, in essence, becomes invisible to the user.

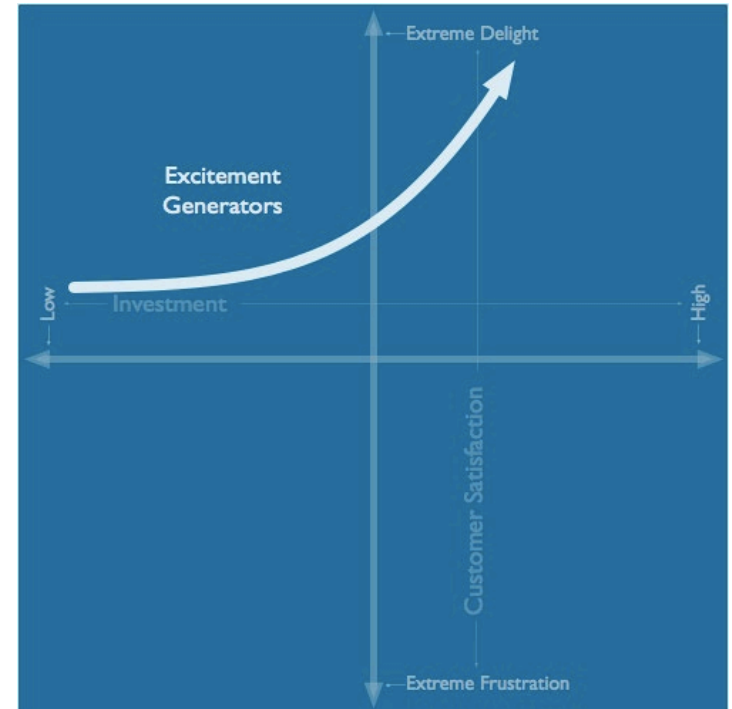
Basic expectations are often compromised by bugs and reliability. When a Skype conversation is cut off by a bad connection, it can frustrate the people on the call. When an e-commerce application double charges the site's shopper, that's frustrating too.



Surprise & Delightful Features

- At the opposite end of the spectrum are features that delight the user. These **score very highly on satisfaction** and in many cases may not take as much investment. Small incremental improvements here have an outsized impact on customer satisfaction.
- Returning to the hotel example, you can see a lot of features that were developed to delight the customer – from the concierge to a complimentary glass of wine on arrival to giving every customer a freshly baked chocolate chip cookie on check-in.
- One of the most challenging aspects of the Kano model is that it predicts that **all features will migrate from delightful to basic expectations**. Once a user has come to expect that delightful feature – whether because you’ve had it for a while or because all their other products have it – the feature has become something they expect.

- Amazon customers report, while they often purchase using the company's 4-5 day free shipping, they **frequently receive their order within one or two days** at no extra charge. This delights the customers, who go on to tell their friends. Amazon has a special relationship with their shippers to accelerate the deliveries. It's part of their desire to "under promise and over deliver."
- **Exploring the users' context and overall experience helps identify potential excitement generators.** These don't have to take a huge Investment.
- Of course, excitement generators won't delight users who are frustrated because basic expectations aren't met. Designers need to ensure they are focusing on both ends of the model.



Apple is strong on Delight but maybe weak on basic (reliability).
How does it deal with this?

Karl Weigers suggests...

- Calibrate this model for your own use with a set of completed requirements or features from a previous product.
- Adjust the weighting factors until the calculated priority sequence agrees with your after-the-fact evaluation of how important the requirements in your test set really were.
- Let Kano influence your settings..
 - Note that for basic expectations the value of not including them (relative penalty) is high whereas the value of including them (relative benefit) is low.
 - For delightful features the value of including them (relative benefit) is high whereas the value of not including them (relative penalty) is low.
- This model can also help you make trade-off decisions when you're evaluating proposed new requirements. Estimate their priority using the model to tell you how they match up against existing requirements, so you can choose an appropriate implementation sequence.

Potential Approach

- Establish Requirements (RD is your guide but not your method)
- Consider Kano Analysis (May result in amendments/additions during the 'Informing' stage...i.e. part of your RD analysis)
- Apply quality criteria (Actually PPQA)
- Apply Relative Weighting (Kano may influence setting of weights) to establish a priority (useful when constraints of budget, time or other resources are considered)
- Establish baseline for engineers to start design and build (Changes now need to be approved....Now in 'Approving' stage)
 - Use agreed change management process to change baselined requirements specification (might use prioritisation table in the decision making around CRs)



COM668 Project

Some Useful Requirements Development & Management Techniques



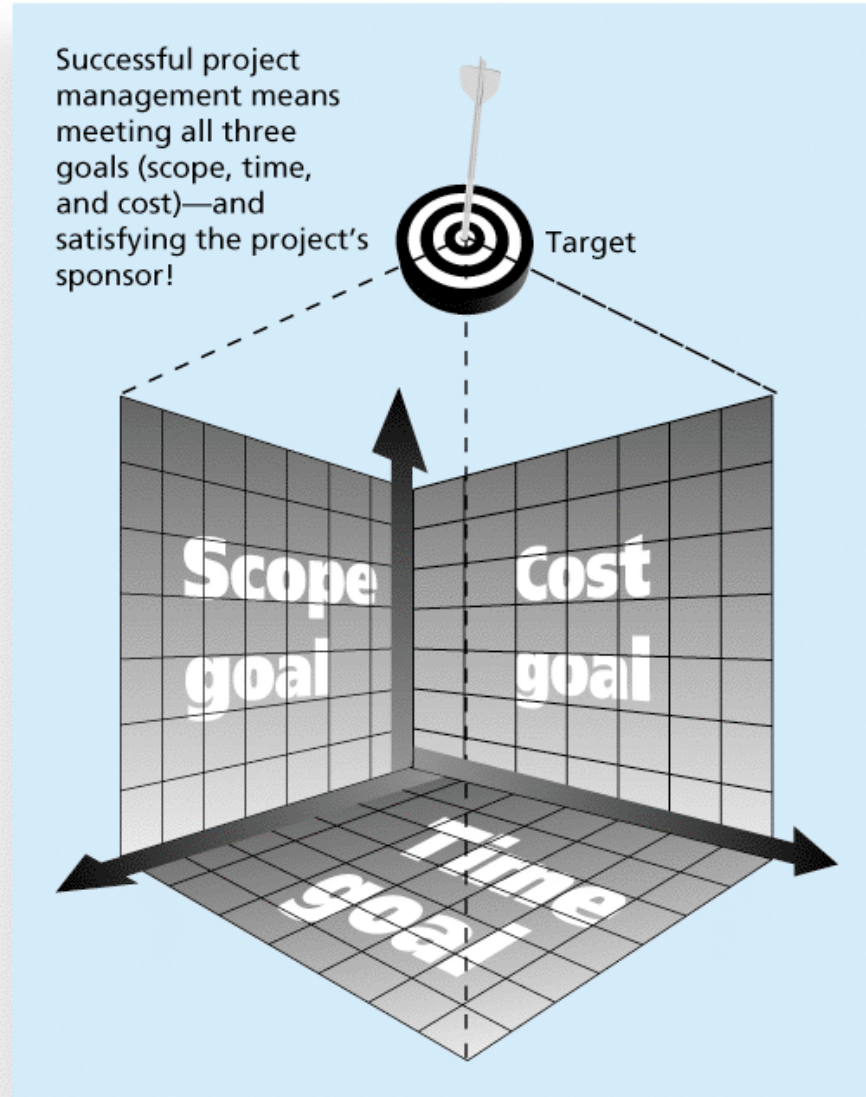
Requirements Development (RD)



Requirement Management (REQM)

Configuration Management (CM)

The Triple Constraint of Project Management



CMMI - REQUIREMENTS MANAGEMENT (ReqM)

The purpose of Requirements Management (REQM) is to manage requirements of the project's products and product components and to ensure alignment between those requirements and the project's plans and work products.

SG 1 Requirements are managed and inconsistencies with plans and work products are identified.

- SP 1.1 Develop an **understanding** with the requirements providers on the meaning of the requirements.
- SP 1.2 **Obtain commitment** to requirements from project participants.
- SP 1.3 **Manage changes** to requirements as they evolve during the project.
- SP 1.4 **Maintain bidirectional traceability** among requirements and work products.
- SP 1.5 Ensure that project plans and work products remain aligned with the reqmts.

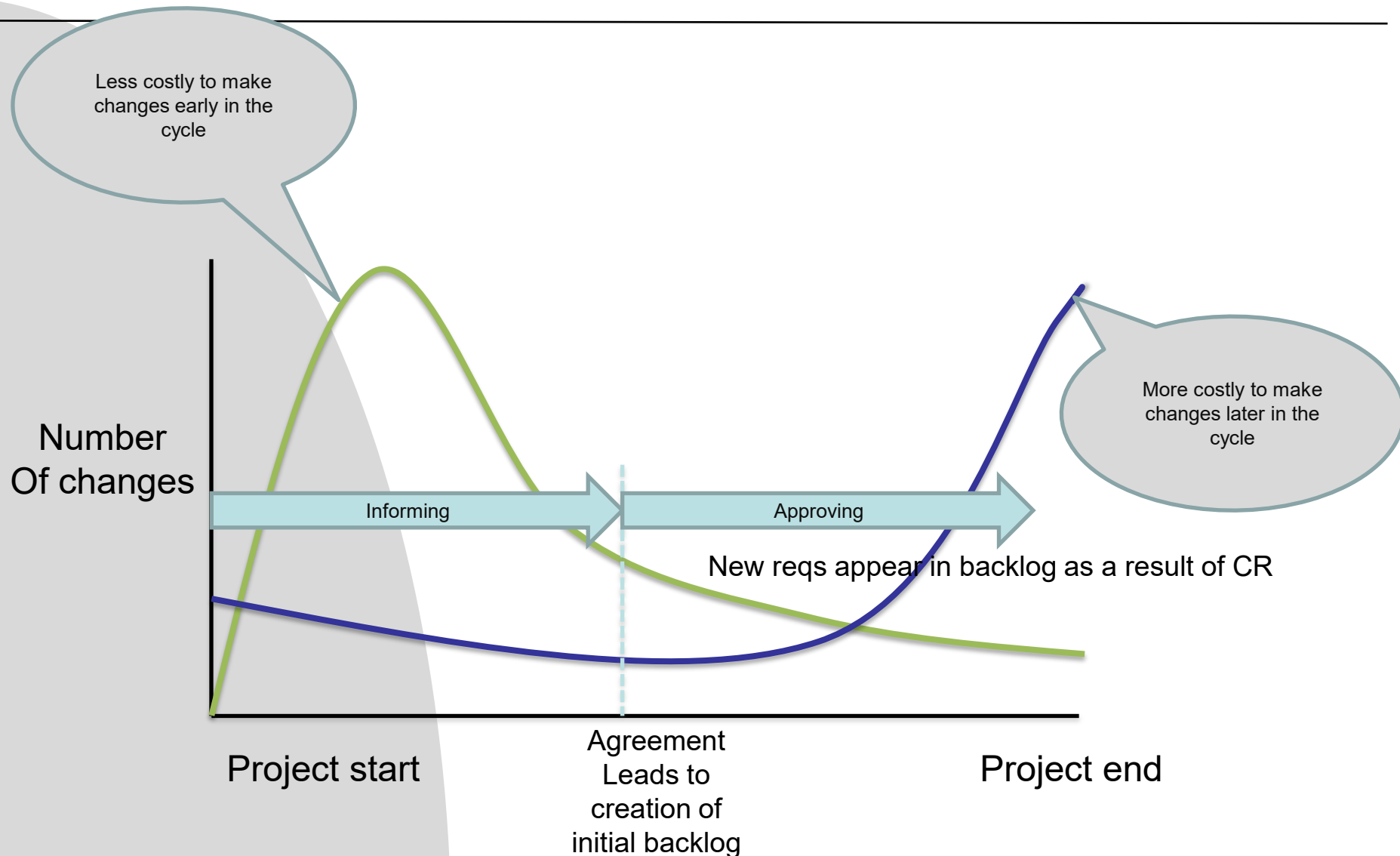
Requirements Management

- REQM is the sum of all activities which take place **after requirements have been specified** (during RD) that are necessary to keep the value of requirements at a high level
- In other words, preventing the requirements specification from becoming less and less valuable/meaningful over time

Traceability

- Requirements should be linked to their **acceptance tests**
- Requirements on **each level of abstraction** should also be linked to the requirements on the level above
 - For example all derived requirements must have a link to the product requirements and all product requirements must have a link to the customer requirements
- Traceability of requirements involves **linking requirements with the work products of other process** areas such as project management, technical solution, risk management and so on.
- Traceability is the responsibility of a Requirements Management Process

Informing & Approval based Change Management



AGILE approaches rely on rapid response to change as a job progresses in the work-flow and/or moves from one sprint to another...

Change Control Form

Section A

Project		Change Number
Controlled Item		Item Version
Identification of Aspect to be Change	For Document give section number / page number For Software give Module, Screen or Report name	
Change Details Include indication of importance and urgency		
Tick if Continued Overleaf ☐		
Requester of Change Print Name		Date Raised

Section B

Investigator of Change		
Impact, give details of other items affected		
Investigation Outcome Reject / Action at No Cost / Action at Cost	Suggested Priority High / Medium / Low	Date Investigated

Section C

Implementor		Date Scheduled
-------------	--	----------------

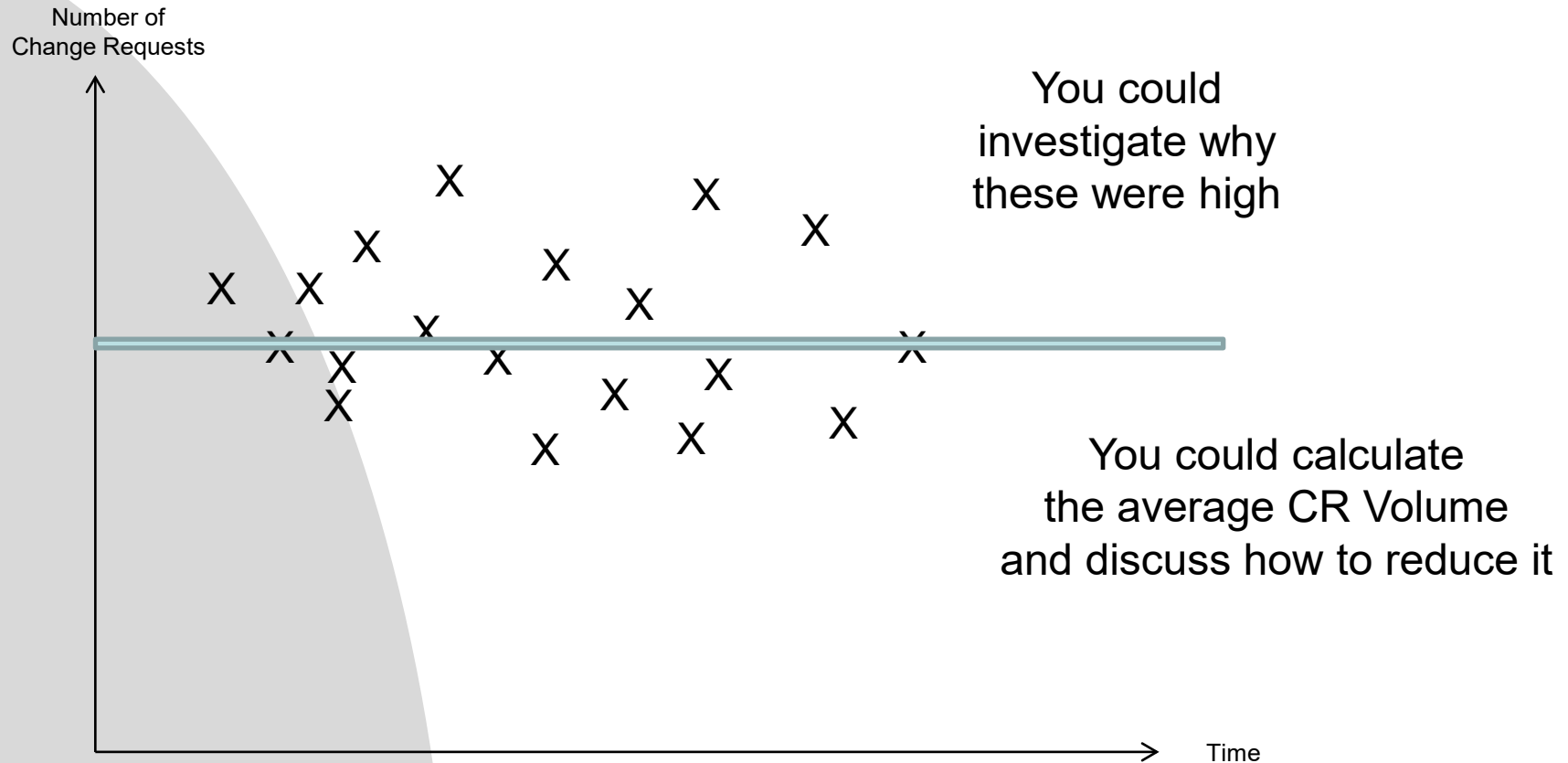
Section D

Change Implemented	Signature	Date
Implementator		
Project Manager		

Change Requests Result in Tickets

[illegible]

What can you start to think about here?



“Requirements Volatility” – this is a key concept to measure in understanding the performance of RD

THE TIMES
THE SUNDAY TIMES
**GOOD
UNIVERSITY
GUIDE
2022**
UNIVERSITY
OF THE YEAR
SHORTLISTED

#WeAreUU

Ulster
University

COM668 Project

Some Useful Requirements Development & Management Techniques



Requirements Development (RD)



Requirement Management (REQM)



Configuration Management (CM)

Configuration Management - CM

To establish and maintain the integrity of work products using configuration identification, configuration control, configuration status accounting and configuration audits

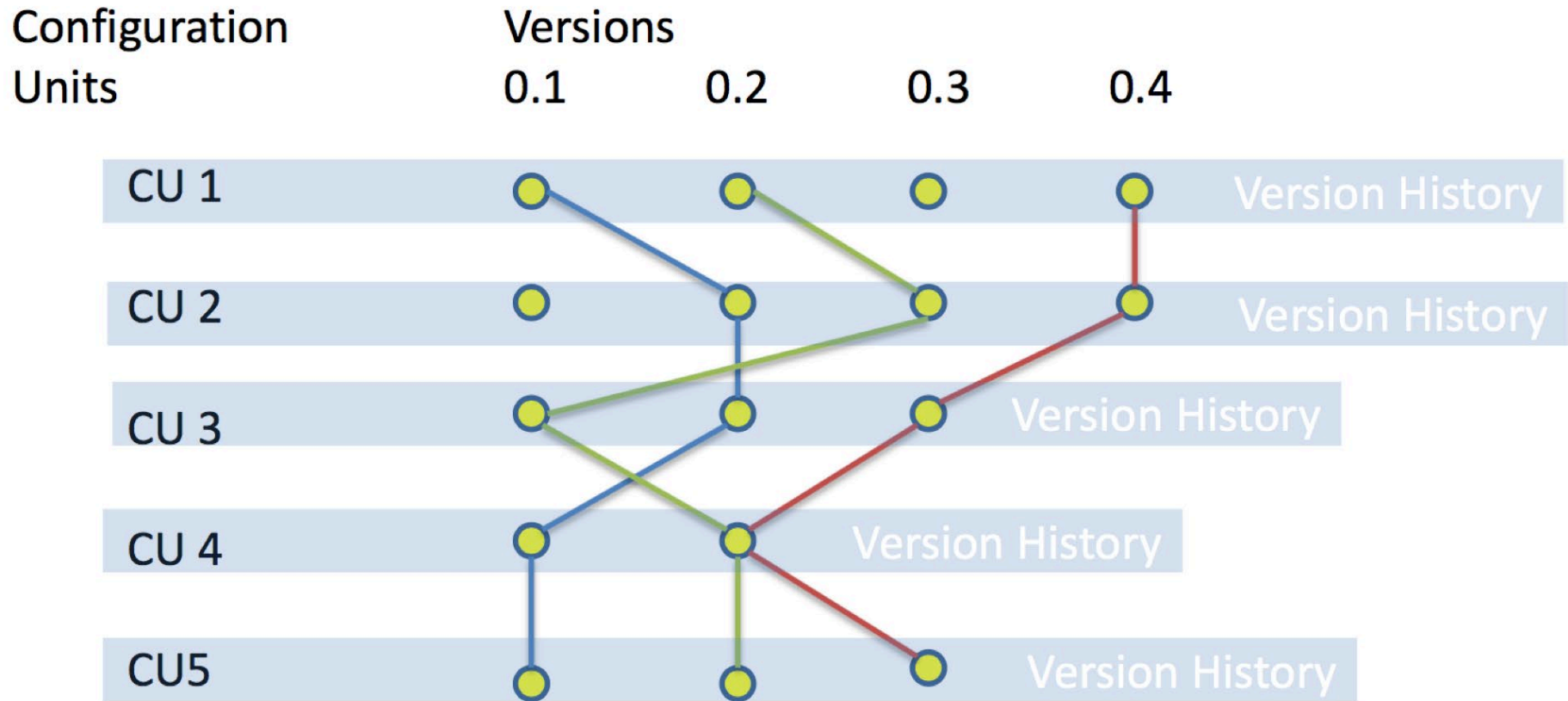
CONFIGURATION MANAGEMENT (CM)

- **SG 1 Baselines of identified work products are established.**
 - SP 1.1 Identify configuration items, components, and related work products to be placed under configuration management.
 - SP 1.2 Establish and maintain a configuration management and change management system for controlling work products.
 - SP 1.3 Create or release baselines for internal use and for delivery to the customer.
- **SG 2 Changes to the work products under configuration management are tracked and controlled.**
 - SP 2.1 Track change requests for configuration items.
 - SP 2.2 Control changes to configuration items.
- **SG 3 Integrity of baselines is established and maintained.**
 - SP 3.1 Establish and maintain records describing configuration items.
 - SP 3.2 Perform configuration audits to maintain the integrity of configuration baselines.

CM Definitions

- **Configuration Unit**
 - Smallest possible building block of a configuration that can be considered atomic. Examples include documents, specifications, code, design artifacts
- **Version**
 - A defined and reproducible state of a configuration unit. Every version has a unique identifier to ensure that configurations can be recreated correctly.
- **History (Change History) (Version History) => Traceability**
 - The version history documents how a configuration unit changes over time.
- **Version Management**
 - System that tracks all versions of a configuration unit (tracks the version history). Changes on an existing version are saved as a new version together with a unique ID, timestamp, userID and other attributes.
- **Configuration**
 - A set of configuration units.

Version and Configuration Diagram



The coloured lines represent the Bill of Materials for various configurations **C1**, **C2** and **C3**

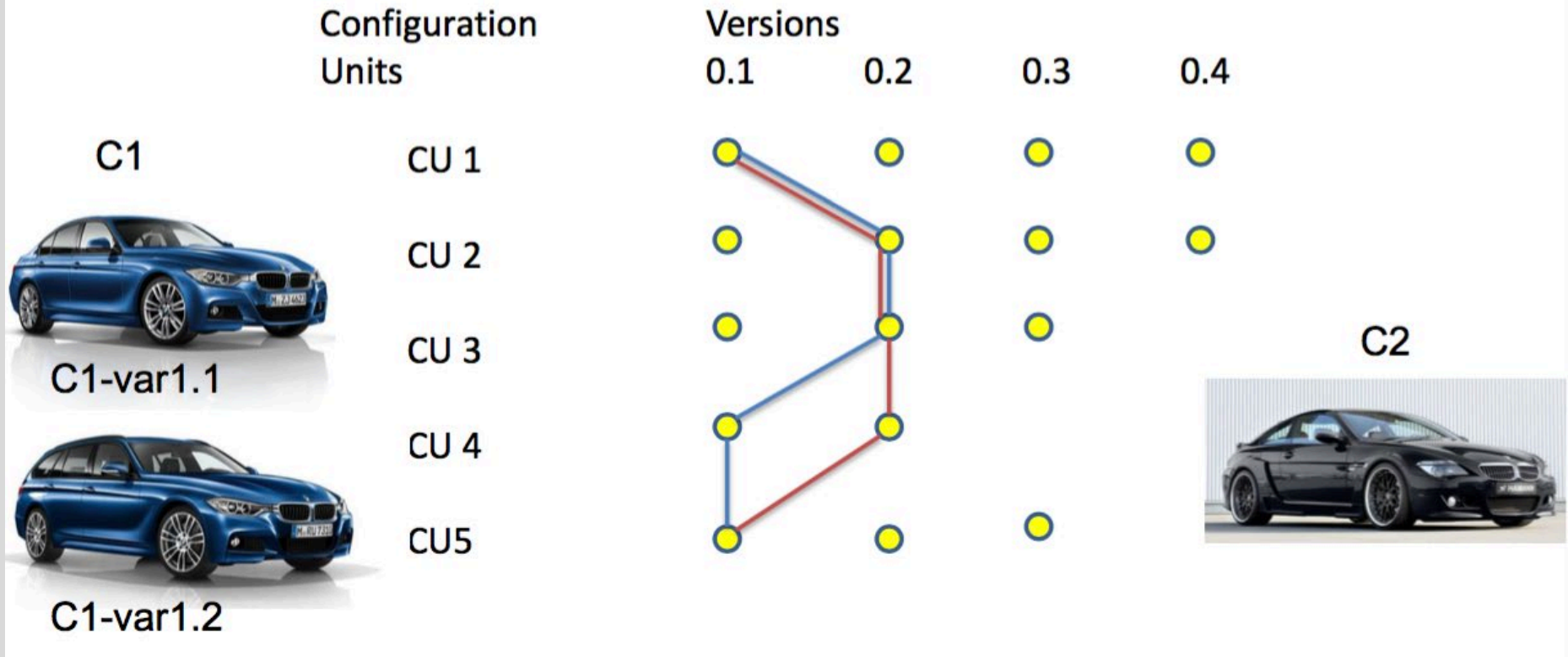
Bill of Material (BOM)

- BOM is the inventory or content of a configuration, consisting of configuration units.
- To ensure that each configuration is assembled from the correct configuration units, the exact identification of the version of each configuration unit within the BOM is necessary. This is the role of Version Management.
- Version Management is a foundation for proper Configuration Management

Version Management Concepts

- **Baseline:** A configuration with a special meaning. A Baseline's content can always be referred to. Baselines are used to control development. A software baseline can be a set of design artifacts, source code files, and executable code, build files and user documentation that have been assigned a unique identifier.
- **Release:** Special form of Baseline. A Release is a Baseline that is made available (maybe to a customer).
- **Variant:** Configurations with similarities in function or content. Variants refer to Configurations. Versions refer to Configuration Units.
- **Branch:** A branch in the linear chain of versions of a configuration unit.
- **Audit:** Confirms that a baseline conforms to a standard and meets its requirements.
- **Build:** A creation of a configuration. A Build is created from a Baseline.

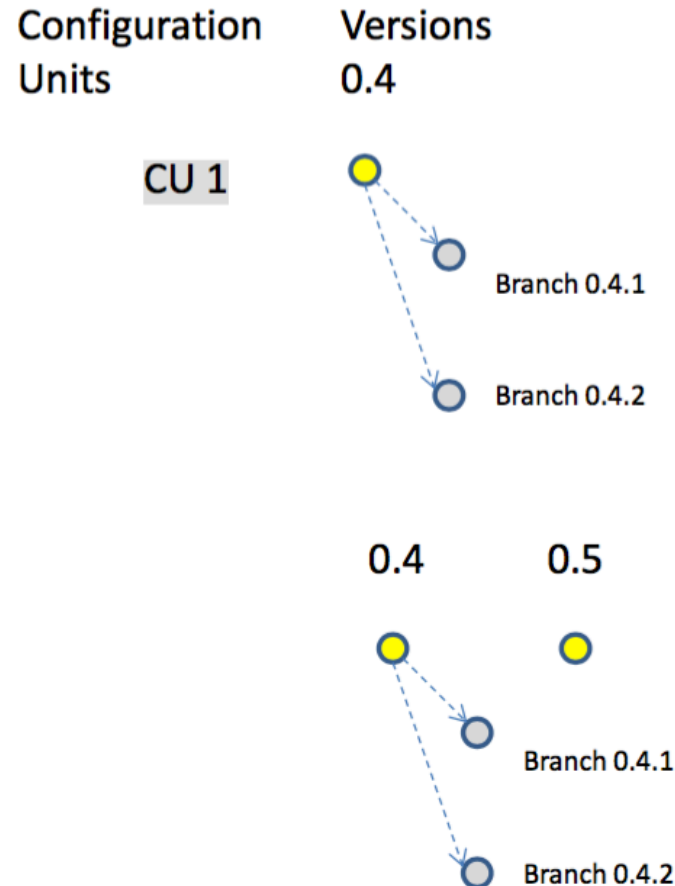
Variants of a Configuration



C1 might have two variants, saloon – (red line) and estate – (blue line) in the diagram. They are not significantly different to be regarded as two configurations. C1-var1.1 and C1-var1.2 as opposed to two separate configurations

Branch in Chain of Versions of a CU

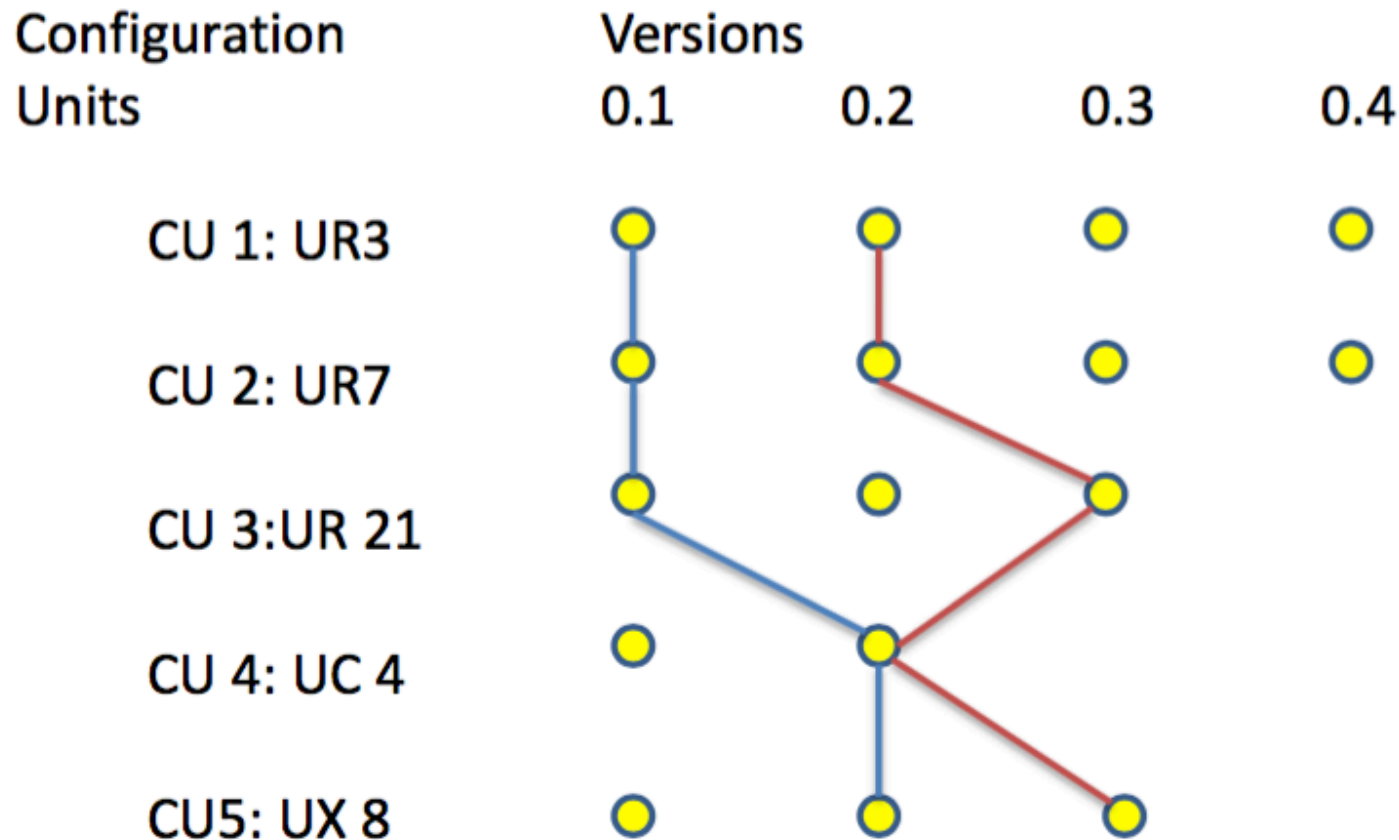
- Branches allow different users/developers to change a given CU at the same time.
- It allows parallel work to be done on a CU.
- The branches can then be Merged into a new version
- This can still be traced in a version management system



Configurations and REQM

- It is possible in CM to treat the entire requirements specification as a configuration unit.
- It is more typical to treat the requirements specification as a configuration. The requirements specification consists of individual requirements and other information.
- A Baseline defines a configuration at a specific time. Baselines of specifications need to be generated (Build).
- Often a requirements baseline is linked to the features for a specific software release.
- Requirements changes (volatility) need to be carefully considered/approved and perhaps scheduled for implementation for a specific product release.

Baseline of a Req. Spec.



Baseline A: Req. Spec

Baseline B: Req. Spec.

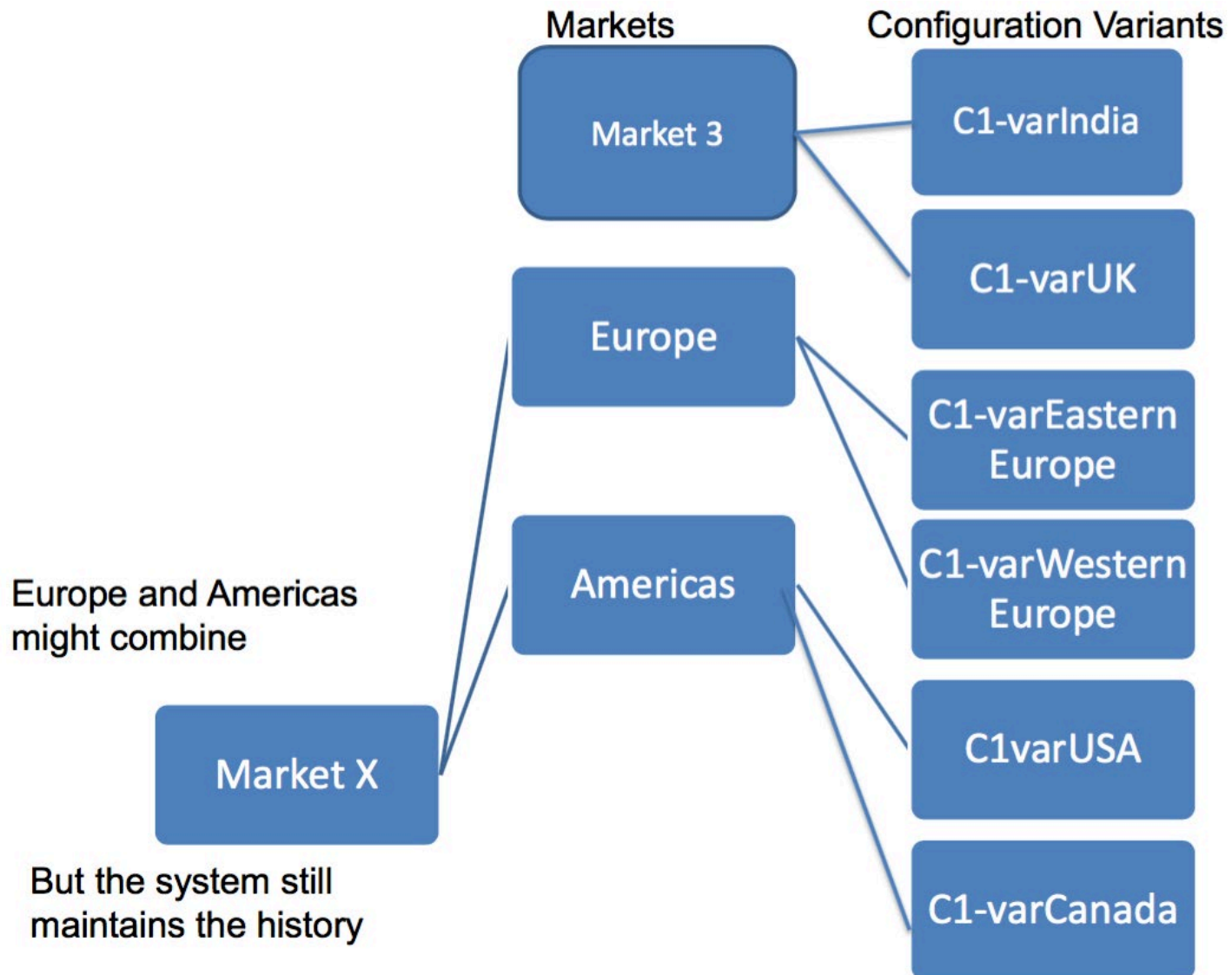
Changing Requirements Specifications in Practice - Versions

- In iterative development there can be multiple baseline configurations.
- Each baseline may refer to multiple versions of a requirement.
- Each version of a requirement that is referred to from a Baseline must be accessible.
 - Baseline B, for example, refers to version 0.2 of CU1:UR3. While examining this baseline, an engineer must be able to access version 0.1 of CU1:UR3. But why?
- A typical situation arises where the change of a requirement must not lead to a system which cannot implement the older requirement.
- Think about new versions of Microsoft Word. The new version must do much of what the old version did.

Changing Requirements

Specifications in Practice - Variants

- Cars are sold in different markets. As there are different regulations in different countries, some details will differ for the various markets. This is a good example of Variants.
- A problem with Variants is that when a Change Request (CR) is received, an affected configuration may have variants. Not all the Variants may be affected.
 - This ***increases the workload considerably***, because it has to be decided which variants are impacted and which are not. The change management is much more complicated.
- If two formerly different markets combine such that their cars are the same, we need to combine two or more variants of a configuration into one. For each CU within the Configuration a decision must be made on whether it remains intact within the Configuration for the new market
 - If it has no variants we either carry it, leave it out or create a variant
 - If it has variants then, which of the alternative variants carry over.
 - It is important to maintain the previous history because the market may resort back to its previous position.
- For some industries the complexity of CM is immense. In the car industry, each individual car may be a unique configuration. Look at the car configurator on BMWs website as an example.
- When a configuration is created, there can be many thousands of configuration units contained in it. Think of all of the various work products that might be contained. Requirements will be among these CUs.



CM versus REQM

- To manage requirements, Configuration Management is needed. CM is a process utilised by REQM
- CM manages the entirety of the CUs in an organisation. Requirements objects are a subpart of these CUs.
 - These Requirements objects (CUs) are the concern of activities within REQM.
- There are two practices of REQM which need CM in particular:
 - SP1.3 Manage Requirements Change
 - SP1.4 Maintain Bi-directional Traceability of Requirements

REQM Tools

- Professional tools for configuration management are the norm.
- Invisible to the user, typically they store only changes between versions using a "reverse delta" mechanism, thereby minimizing the required memory. The reverse delta algorithm stores only the latest version as a whole.
- For all previous versions, only the difference (delta) is stored. By applying all earlier deltas to the current version, any past version can be reconstructed.

Project Management (PMBOK) Perspective

Table 4-3. Suggestions for Performing Integrated Change Control

View project management as a process of constant communication and negotiation.

Plan for change.

Establish a formal change control system, including a change control board (CCB).

Use effective configuration management.

Define procedures for making timely decisions on smaller changes.

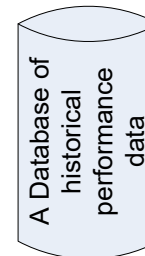
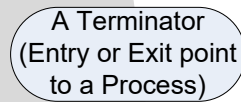
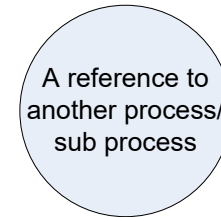
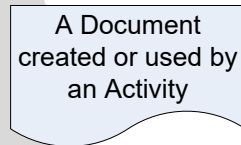
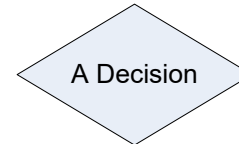
Use written and oral performance reports to help identify and manage change.

Use project management and other software to help manage and communicate changes.

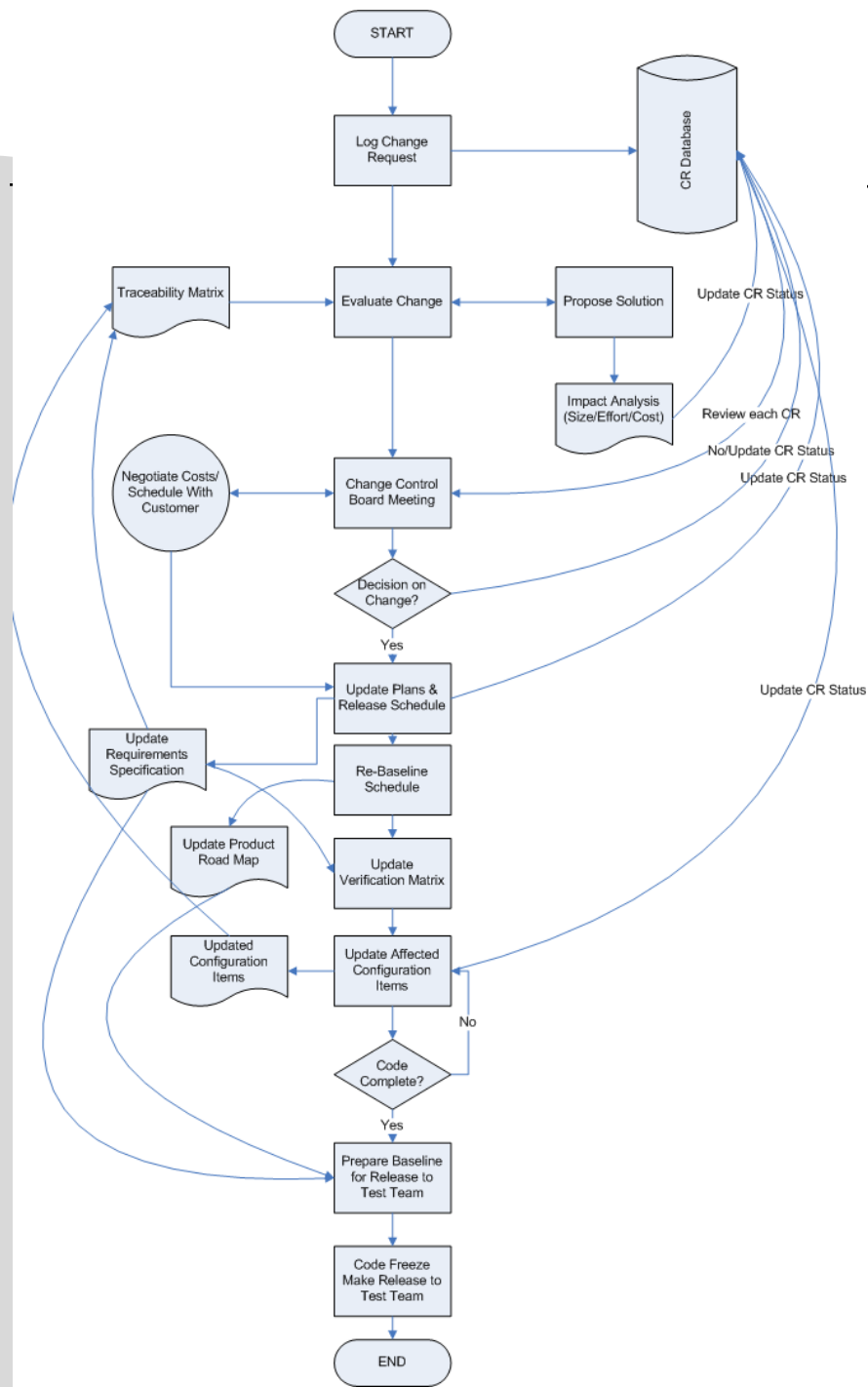
Focus on leading the project team and meeting overall project goals and expectations.

An Example Change Management Process

Notation Key



A Change Management Process



References/Bibliography

- Weigers (1999) - Karl Weigers, “First Things First: Prioritizing Requirements”, Software Development (1999), <http://www.processimpact.com/articles/prioritizing.html>
- Weigers (2013) Weigers, Karl, “Software Requirements”, 3rd Edition, ISBN-13: 978-0735679665 (August 2013)
- Kano (1984) – Noriaki Kano, Seraku Nobuhiku, Takahashi Fumio and Tsuji Shinichi (April 1984), “Attractive Quality and must-be quality”, Journal of the Japanese Society for Quality Control, Vol 14, No 2, pp39-48.
- “What is the Kano Model”, www.kanomodel.com

Video Closed Captions

Please note that this video has been produced with automatic closed captions for accessibility purposes. As with any automated speech recognition system there may be some errors in the speech to text conversion process.

In most cases these will be minor and won't affect the overall learning for those who use closed captions, but if any clarity is required or if any errors in closed captions need correcting please do contact your tutor to make them aware.